```
$   echo "Visit http://127.0.0.1:8080 to use your application"
Visit http://127.0.0.1:8080 to use your application
~/book/pact-js-example on main*
$   kubectl --namespace default port-forward $POD_NAME 8080:$CONTAINER_PORT
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

Figure 9.18 Kubernetes output, showing the Pact Broker URL

Navigating to `http://localhost:8080/` should display the Pact Broker homepage, as shown earlier in figure 9.15.

9.5 SaaS vs. self-hosted Pact Broker

Now, we've been through the setup for both the SaaS version of Pact Broker in PactFlow and the open source version of Pact Broker. Reflecting on each Pact Broker implementation, we'll look at the pros and cons. Some key considerations include security, maintenance, and exclusive features.

As we saw in section 9.3.2, PactFlow users can authenticate using an API bearer token. The token can be easily rotated and revoked if required. However, the open source version of Pact Broker shown earlier in listing 9.3 supports username and password as credentials only. The credentials for Pact Broker were set up upon launching Pact Broker.

Another important factor that teams often weigh up is the possible maintenance effort. As shown in section 9.4.2, deploying the open source Pact Broker with Kubernetes could help stabilize it, but it still requires some maintenance. Pact Broker will require security patches and dependency updates. Support is well represented by the open source community. However, paying for a hosted version comes with the peace of mind that dependencies are kept up-to-date. In addition, support is provided by the PactFlow team to help when needed. Weighing up the maintenance costs against the costs of the SaaS is another important factor to consider.